



Software Engineering Department

Capstone Project - Phase B

Personalized Smart Cart with AI Assistance

25-2-D-23

Authors

Tomer Rotman
`tomer.rotman@e.braude.ac.il`

David Zorin
`david.zorin@e.braude.ac.il`

Supervisor

Uzi Rosen
`rosen@braude.ac.il`

Github Repository

[Link](#)

Simulation Video

[Link](#)

Table of Contents

1	Introduction.....	1
2	Research Process Summary	3
2.1	Allergies and Dietary Needs Literature Review Summary	3
2.2	Existing Smart Cart Solutions Limitations Summary	4
2.3	Theoretical Background Summary	4
3	General System Description.....	5
3.1	System Capabilities.....	5
3.2	Stakeholders Benefits.....	6
3.3	Operational Flow	7
4	System Architecture.....	8
4.1	Client Interfaces.....	9
4.2	Local Controllers	9
4.3	Cloud-Hosted Backend	9
4.4	External Services	10
5	Hardware Integration Process.....	11
5.1	Hardware Components Selection	11
5.2	Physical Enclosure Design	12
6	Software Development Process	13
6.1	Development Methodology.....	13
6.2	Engineering Adjustments.....	14
6.3	Implementation Overview	15
6.4	Development Challenges.....	18
7	Results.....	19
8	Evaluation and Conclusions	19
8.1	Evaluation	19
8.2	Conclusions.....	20
9	Lessons Learned.....	21
10	References	21
11	Appendices.....	24
11.1	User Manual	25
11.2	Maintenance Guide.....	34

Abstract

This project presents a Raspberry Pi-based smart cart system with a React frontend and FastAPI backend, integrated with an AI-driven voice assistant powered by VAPI. The assistant responds to user inquiries regarding nutritional information, suggests alternative products based on specified needs for a scanned item, and visually displays product locations on an interactive supermarket map. The system automatically alerts users when scanned products conflict with allergies or dietary preferences and recommends appropriate alternatives, reducing the need for manual product searches. In addition, users can create shopping lists in advance using personal devices and upload them to the system, which generates an optimized pick-up route using OR-Tools and NumPy. To further reduce the likelihood of forgotten items, the system provides suggestions for frequently purchased products during checkout. Altogether, these features enable a more personalized, informed, and time-efficient shopping experience.

1 Introduction

Despite the rapid growth of online shopping, physical grocery stores continue to be the primary choice for many consumers around the world. In fact, 84% of all retail sales still occur in physical stores, partly due to the limitations of online shopping such as lack of immediacy (since customers must wait for delivery), uncertainty (as products cannot be seen or touched before purchase) and perceived risks (such as receiving items that do not match expectations) [1]. One of the most persistent challenges in traditional grocery shopping, however, is the problem of lengthy checkout lines, particularly during peak shopping periods such as weekends and holidays. Negative emotional responses to long waiting times have been shown to worsen the perceived store image [2], which refers to the overall perception that consumers form about a retail store. Since store image significantly influences customer loyalty [3], retailers are encouraged to seek innovative solutions in order to retain customer loyalty and expand their customer base.

Over the past two decades, the expansion of the Internet of Things (IoT) has driven a major technological revolution across multiple industries. In the retail sector, smart carts represent a specific application of IoT. These systems typically include a touchscreen, barcode scanners, and network connectivity, enabling them, through the touchscreen interface, to display the names and prices of the items selected by the consumer, along with the total cost of the purchase. When equipped with a card terminal, smart carts allow customers to complete payments directly at the cart – eliminating the need to wait in checkout lines. Even in the absence of integrated payment functionality, smart carts have the potential to eliminate queue times by removing the need to scan items at checkout. In such cases, checkout lines serve primarily as a verification stage to prevent theft, typically supported by weight sensors embedded along the checkout path to detect discrepancies between declared and actual items. As smart shopping carts offer cost-cutting potential for retailers and an improved shopping experience for consumers, the global market is projected to grow at a compound annual growth rate of 27.48%, reaching USD 9.74 billion by 2030 [4].

However, the grocery shopping experience remains time-consuming and frustrating due to several factors, many of which current smart cart solutions could address, yet often fail to resolve effectively. One persistent challenge affects consumers with dietary restrictions, such as allergies, who must frequently pause to examine ingredient lists and nutrition labels, significantly slowing their shopping process. Another common issue is the lack of route efficiency: consumers typically pick up products in a scattered way, resulting in unnecessary backtracking. This inefficiency becomes even more pronounced when consumers are unfamiliar with the supermarket layout or when certain products are either temporarily out of stock or simply not sold in that supermarket, leading to additional time spent trying to locate them. In such cases, when a product cannot be found, consumers often have to start a second search - this time for a staff member who can help. For some consumers, when one is finally found, the interaction itself may be uncomfortable or inconvenient [5]. Lastly, even after completing their shopping, consumers often realize they have forgotten to purchase essential items, leading to repeated visits to the supermarket.

An effective solution must aim to eliminate, or at least reduce, the likelihood of these factors arising during the shopping process. Addressing some of these challenges requires an advanced form of support that can adjust to consumer needs in real-time. This can be achieved through personalization, which has emerged as a key strategy for improving customer experiences across industries, particularly in retail environments. Personalization can be understood as the process of modifying a system's functionality, interface, content, or information access in order to make it more personally relevant to the user or a group of users [6]. Typically, personalization efforts have relied primarily on static data, such as purchase histories. However, recent advances in large language models (LLMs) have made it possible to create AI assistants that dynamically adapt system behavior by understanding their evolving preferences [7].

Building on these advancements, this project proposes a smart cart system integrated with a digital companion, designed to provide on-demand, tailored product support through intuitive, voice-based interactions. In addition to the companion's AI-driven capabilities, the system also features complementary functions - such as route guidance and innovative recommender subsystems. Together, these capabilities aim to set a new standard for what smart retail technology can offer.

2 Research Process Summary

As part of Phase A, the research process began with an examination of challenges encountered during grocery shopping, as generally described in the introduction. This analysis established the set of needs that the proposed smart cart system should address.

Building on this understanding, existing smart cart solutions were reviewed to evaluate their typical capabilities, modes of operation, and practical constraints, including usability considerations and development complexity. This review also enabled an assessment of which challenges in the shopping experience are effectively addressed by current smart cart solutions and which remain insufficiently supported.

Based on the identified gaps, a system concept was formulated. This, in turn, required additional theoretical background research to understand the relevant technological foundations.

2.1 Allergies and Dietary Needs Literature Review Summary

A focused investigation of allergies and dietary needs during the research process revealed the scale and relevance of this challenge in grocery shopping.

Food allergies are no longer a marginal concern. Their prevalence has increased significantly over recent decades and is frequently described as an “epidemic” [9]. Given that allergen exposure can lead to severe and potentially life-threatening reactions [8], this trend highlights dietary safety as a critical issue.

In addition to allergies, a growing number of consumers follow health-related or ethically motivated dietary patterns that require careful assessment of product ingredients and nutritional composition in order to make suitable purchasing decisions [10,11,12].

The reviewed literature indicates that consumers rely primarily on food labels. However, these are often difficult to interpret due to dense information and small fonts [13], making this a key factor contributing to increased shopping time and frustration.



Figure 1. A consumer reviewing a nutrition label, which is noted for its small font size and density.

2.2 Existing Smart Cart Solutions Limitations Summary

A review of existing commercial smart cart systems conducted in Phase A exposed several practical limitations that directly informed the design of the proposed system. Personalization is largely promotion-oriented, and product location guidance, when available, neither reflects real-time availability nor supports low-effort interaction, as it typically relies on text-based input, which can reduce its practical usefulness for certain user groups.

2.3 Theoretical Background Summary

2.3.1 Traveling Salesman Problem

Determining an efficient pickup order for products in a supermarket can be modeled as a Traveling Salesman Problem (TSP), a classical problem in computer science in which a set of locations must be visited exactly once while minimizing total travel distance [14,15]. The TSP is NP-hard, meaning that finding an optimal solution becomes computationally infeasible as the number of locations grows due to the factorial increase in possible routes. Consequently, practical applications do not rely on exact optimization but instead employ heuristic or approximation methods that produce near-optimal solutions within reasonable time constraints, which is sufficient for real-world scenarios where responsiveness is prioritized over guaranteed optimality.

2.3.2 Large Language Models

Large Language Models (LLMs) are deep learning models trained on large-scale textual data to capture semantic and contextual relationships in natural language, enabling them to generate responses that give the impression of human-like understanding and making them suitable for both interactive and task-oriented applications [16]. Despite their capabilities, LLMs are computationally demanding: response latency depends on available computational resources, which consume substantial energy and lead to increased operational cost [17,18]. Furthermore, LLMs are based on the Transformer architecture [19], which generates outputs probabilistically by predicting the next token given the preceding context, meaning that small changes in input wording can result in significantly different outputs. In practice, careful prompt design is therefore used to constrain model behavior, guide response structure, and reduce unnecessary computation, improving reliability in real-time applications [20,21].

2.3.3 Voice Assistants

Voice assistants are software agents that process human speech and respond using synthesized audio output in order to answer questions or perform tasks, offering users a hands-free and intuitive interaction experience in a wide range of applications [22]. These assistants rely on a processing pipeline that combines Speech-to-Text (STT), natural language understanding, and Text-to-Speech (TTS) components. The STT stage converts raw audio input into text by analyzing acoustic features and applying language models to resolve ambiguities in speech, while the TTS stage performs the inverse operation by transforming generated text into synthesized speech [23,24]. Modern voice assistants integrate Large Language Models as the natural language understanding component, enabling support for open-ended questions rather than fixed command sets [25].

Voice-based systems must operate under real-time constraints to maintain user trust, as excessive latency or misinterpretation can be perceived as system failure.

2.3.4 Recommender Systems

Recommender systems suggest items (such as products or services) to users by estimating their preferences based on available data about them [26]. These systems are beneficial for retailers since they can enhance customer loyalty [27]. Various recommendation techniques exist, each relying on different sources of information. Two of the most significant challenges associated with recommender systems are the cold-start problem, which arises when insufficient data is available to generate personalized suggestions for new users, and gaining user trust, as users are less likely to accept recommendations that are poorly justified.

3 General System Description

3.1 System Capabilities

As a smart cart system, the proposed solution supports the baseline functionality commonly found in commercial smart carts. This includes identifying products as they are added to the cart through barcode scanning, maintaining a real-time list of selected products, updating quantities, and continuously displaying the accumulated cost during the shopping session.

Beyond these standard functions, the proposed system introduces several advanced capabilities aimed at addressing the factors identified during the research process that continue to contribute to inefficiency and frustration in grocery shopping. Each capability is described below, together with its role in reducing the impact of these factors.

3.1.1 AI-Based Voice Assistant

Through natural language voice interaction, the assistant enables an intuitive and engaging form of interaction, allowing consumers to inquire about product-related information. Specifically, the assistant supports the following capabilities:

- Answering questions about the nutritional content and ingredients composition of products added to the cart
- Reactively providing alternative product suggestions based on inferred criteria
- Automatically detecting conflicts between added product information and inferred dietary restrictions
- Proactively suggesting suitable alternatives when such conflicts are detected
- Presenting a product location within the supermarket layout, while verifying availability

This capability reduces the need for manual label inspection or unnecessary wandering within the supermarket to locate products, all without relying on manual text input. By providing personalized suggestions, the voice assistant operates as a recommender subsystem.

3.1.2 List-Based Shopping Guidance

The system provides list-based shopping guidance. This capability combines:

- Tracking the shopping list throughout the session and alerting the user when products from the list have not added to the cart upon checkout
- Guiding the user step by step through an efficient in-store pickup route

The shopping list can be prepared in advance using a dedicated web application and loaded onto the smart cart at any moment during the shopping session.

This capability ensures that users complete their shopping list without forgetting products while simultaneously reducing unnecessary walking and backtracking within the supermarket.

3.1.3 Potentially Forgotten Products Suggestions

Another recommender system. During checkout, the system provides suggestions for products that the consumer may have forgotten to add to the cart, specifically products that the consumer frequently purchases, based on his historical purchasing behavior.

This capability reduces the likelihood of forgetting to buy products that were not explicitly included in the shopping list, minimizing additional shopping trips caused by incomplete purchases and thereby improving overall convenience.

3.2 Stakeholders Benefits

3.2.1 Consumers

As the primary users, consumers benefit not only from a more efficient shopping experience but also from a fundamentally more informed one, promoting long-term behavioral change toward healthier and more mindful consumption.

3.2.2 Retailers

As those responsible for shaping the consumer experience, retailers stand to benefit significantly from the integration of such a smart cart system, as it offers two key avenues for increasing revenue:

- As the smart cart enables more satisfying and efficient shopping journey, it can attract a broader customer base - translating to more foot traffic and potentially higher overall sales.
- The recommender systems play a strategic role by reducing product abandonment. When a product does not meet the consumer's needs or preferences, the system suggests suitable alternatives, ensuring that the purchase opportunity is retained. This is intended to reduce the likelihood of product drop-off rates and, therefore, boosting sales.

3.2.3 Manufacturers and Brands

Food manufacturers and brands can leverage the smart cart system as an opportunity to remain competitive and adapt to shifting consumer behavior. Products that meet a wider range of user preferences may gain increased visibility through the alternative product suggestions module, creating new opportunities for brand distinction and expanded market reach.

3.3 Operational Flow

Following the description, a simplified overview of the smart cart system's operational flow during a typical shopping session is presented below. It highlights the main user actions and system responses.

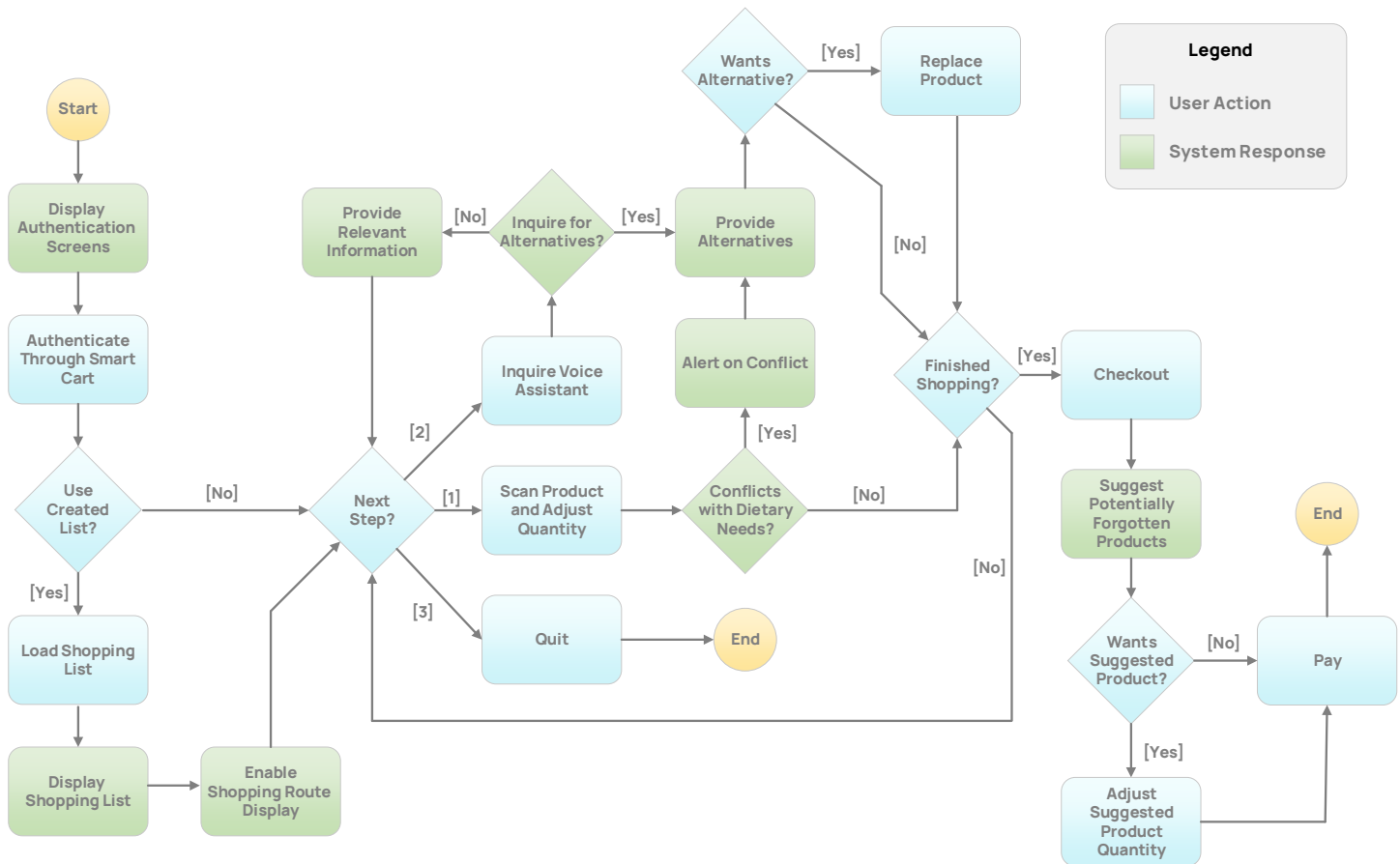


Figure 2. High-level system flow of the smart cart during a shopping session.

4 System Architecture

The initial system architecture was formulated during Phase A as part of the engineering process, based on the system characterization, identified limitations and challenges, and expected achievements defined at that stage. During Phase B, the architecture was further refined to address minor inaccuracies, additional constraints, and design considerations that emerged during development. The final architecture is presented in Figure 3.

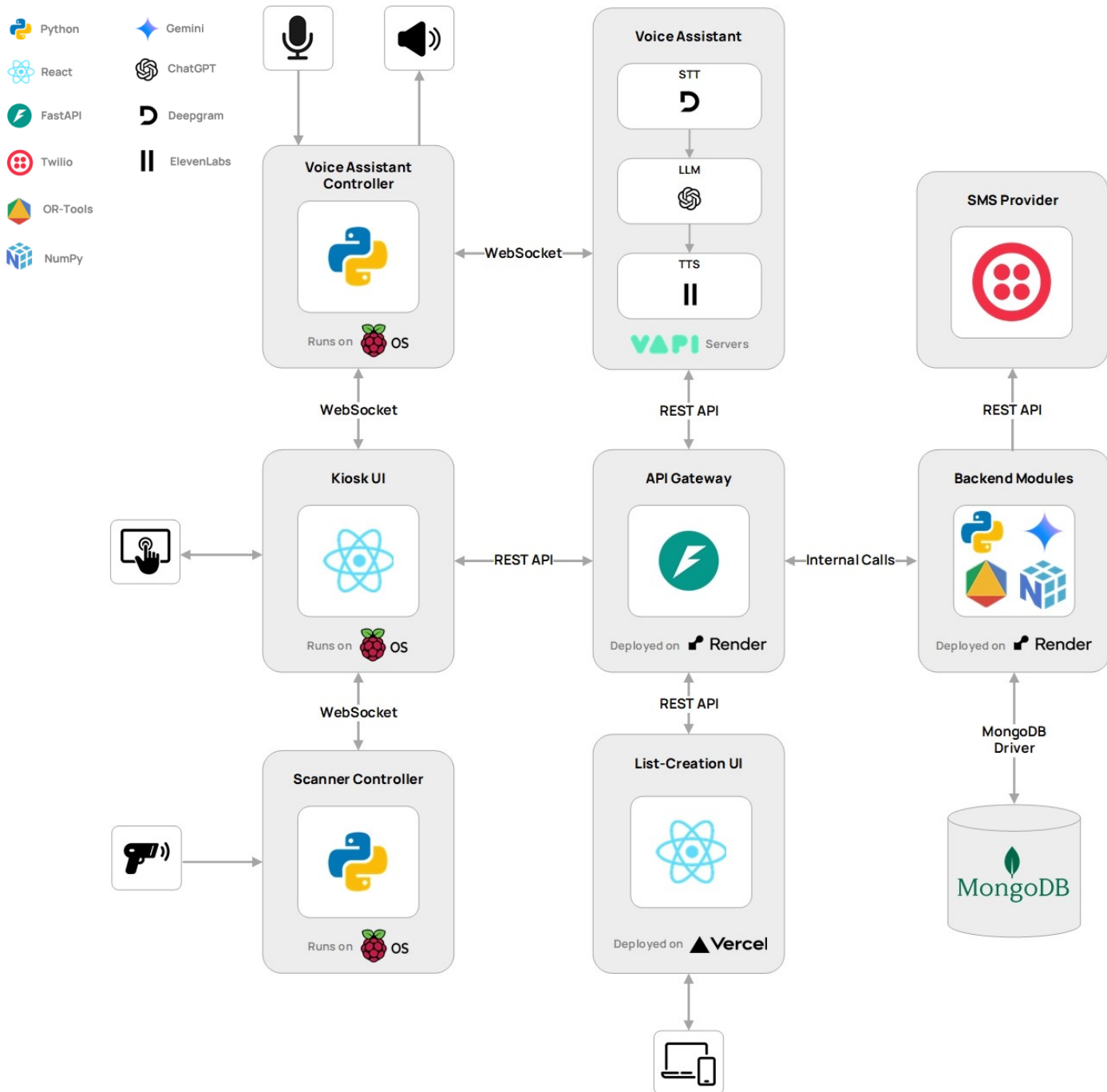


Figure 3. The smart cart system software architecture.

4.1 Client Interfaces

The interfaces through which the consumers interact with the system:

- **Kiosk UI** – Touch-based interface running on the cart display. Responsible for the shopping session flow.
- **List-Creation UI** – Runs on the consumer personal device for preparing a shopping list in advance and upload it to the system.

Both interfaces are built using **React**. For the list-creation interface, React was selected due to its widespread adoption in modern web application development. However, for the Kiosk UI, React was chosen because it enables the user interface to run within a standard browser environment, and allowing the interface to be deployed on different operating systems, if required in the future.

4.2 Local Controllers

The local controllers are services that operate on the single-board computer, **Raspberry Pi**, which serves as the smart cart computing unit:

- **Voice Assistant Controller** – Manages the audio devices and handles communication between the Kiosk UI and the external voice assistant service.
- **Scanner Controller** – Manages the barcode scanner device, processes scan events, and forwards the information to the Kiosk UI.

Both controllers are implemented in **Python**, as it enables direct access to peripheral devices, supports asynchronous communication, and simplifies development.

While regular sockets would be sufficient to exchange events data, they are not compatible with browser-based applications. Therefore, communication between the local controllers and the kiosk UI is performed using **WebSocket**, enabling real-time updates.

4.3 Cloud-Hosted Backend

The backend provides centralized access to the system's core business logic and data management. It consists of:

- **API Gateway** – Exposes the system's functionality through a set of RESTful endpoints. It handles requests originating for the client interfaces and the external services, and routes them to the appropriate backend services.
- **Backend Services** – A collection of internal services that implement the system's domain logic. They manage shopping session functionality and handle the persistence and retrieval of data required by the system, with each service operating independently and encapsulating a specific functional responsibility.
- **Database** – A persistent storage used to maintain data such as user information, product information, purchases history and more. Centralized data storage enables consistent behavior across multiple client interfaces.

The backend services are implemented in Python, selected for its extensive ecosystem of libraries that support optimization and vectorized computation, such as **OR-Tools** and **NumPy**, which are utilized for route-related functionality. In addition, Python is widely adopted in machine learning development, making it suitable for potential future extensions.

Since the backend services are implemented in Python, **FastAPI** framework was selected to create the API, due to its native support for asynchronous request handling, which is well suited for serving concurrent requests from multiple smart carts.

The backend is hosted on **Render**, a managed cloud hosting platform that provides native support for FastAPI deployments. Render was selected due to its availability of a free-tier offering, which is suitable for prototyping and demonstration purposes within the scope of this academic project.

Unlike traditional relational databases, NoSQL databases offer greater flexibility, making them well-suited for fast prototyping and iterative development. **MongoDB** was chosen due to its support for organizing data into collections, which function similarly to relational tables and allow for more structured data modeling.

To interact with MongoDB, the system uses **Beanie**, an object–document mapper that combines MongoDB’s asynchronous Python driver with **Pydantic**-based schema definition and validation. This approach enables structured and type-safe data modeling while preserving the flexibility of a NoSQL database. In addition, Beanie’s native asynchronous support allows efficient, non-blocking database operations.

4.4 External Services

External services are integrated into the system to support capabilities that require specialized infrastructure or computational resources:

- **VAPI** – A web-based platform used to deploy AI-based voice assistants, providing access to a variety of speech processing and language models. The platform enables the definition of a system prompt, which governs the assistant’s behavior and response style, as well as the specification of callable tools that allow the assistant to retrieve and store data through interactions with the backend via webhook.
- **Twilio** – A third-party communication service used to support user authentication. The service delivers SMS messages containing one-time password (OTP) verification codes generated by the backend, allowing users to access the client interfaces without the need to create or manage persistent credentials.

VAPI was selected due to the availability of a free-tier offering, which was sufficient for development within the scope of the project. The platform provides built-in logging and monitoring capabilities that enable inspection of interaction flows and analysis of unexpected system behavior. In addition, VAPI supports cost tracking, which is important for assessing the economic feasibility of integrating a voice assistant into a smart cart system.

Twilio was selected similarly due to the availability of a free-tier offering. Although the free-tier includes only a single verified phone number, it was sufficient to test that the authentication logic works.

5 Hardware Integration Process

Hardware integration was performed in parallel with the software development process throughout Phase B. Phase A focused on identifying the hardware required for the implementation of the smart cart system and on selecting suitable computing unit. However, the selection of **specific** peripheral components and the finalization of the physical enclosure design were carried out during Phase B.

5.1 Hardware Components Selection

During Phase A, the functional and computational requirements of the smart cart system were analyzed, leading to the selection of a suitable computing unit. Based on these requirements, the Raspberry Pi single-board computer was chosen as the system's core platform, due to its ability to run a full operating system, support a high-resolution graphical user interface, and interface with a wide range of peripherals through standard interfaces, in particular USB and HDMI, which are the interfaces used by the selected peripherals, simplifying integration. The peripherals are listed below.

- **JOHNWILL 11.6-inch Touchscreen Display**

The chosen screen size provides sufficient surface area to comfortably present multiple visual elements simultaneously, thereby improving usability. This enables a layout in which the cart view is always visible, alongside either the shopping list together with the supermarket map, or alternatively the chat-based interaction history. In addition, the display offered a favorable cost-to-size ratio at the time of purchase.

- **Sycreader U12 2D Wired Barcode Scanner**

A 1D scanner is sufficient for reading standard product barcodes. However, a 2D scanner also supports scanning QR codes, thereby enabling future system extensions. For example, it allows authentication mechanisms based on QR codes generated by the backend, which could replace SMS-based authentication and reduce operational costs. A wired scanner was preferred over a Bluetooth alternative to avoid the need for managing an additional battery, which would complicate charging logistics in real deployment.

- **Seed Studio ReSpeaker Lite 2-Mic Array**

This component consists of two microphones and is designed to capture speech from a distance of up to three meters, which is essential in a smart cart scenario where users cannot be expected to speak directly into a microphone. For this reason, this microphone array is also commonly used in other voice assistants' applications. This microphone array also includes Acoustic Echo Cancellation (AEC) algorithm, which is intended to reduce feedback caused by the system's own audio output during voice interactions. Although this capability was not explicitly tested during development, it provides a foundation for future enhancements of voice interactions, such as handling user interruptions while the assistant is speaking.

- **Seed Studio Mono Enclosed Speaker**

This speaker connects directly to the selected microphone array and was found to provide sufficient audio clarity for voice-based feedback. Although the touchscreen display includes built-in speakers, they are positioned at the rear of the screen, which is unsuitable for the planned physical enclosure, since this would have resulted in degraded audio delivery to the user.

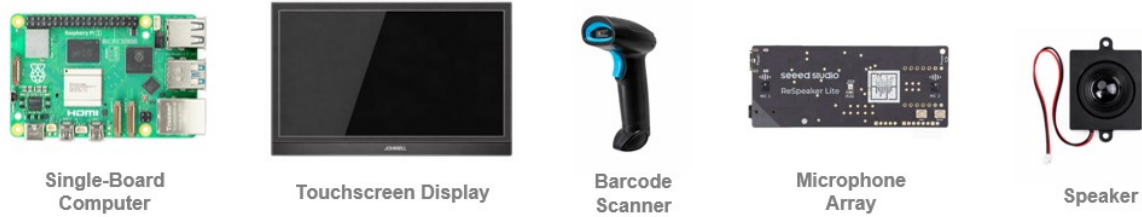


Figure 4. *Hardware components integrated into the smart cart system.*

5.2 Physical Enclosure Design

Once the peripheral components were received, their physical dimensions could be measured accurately. This enabled planning of the overall enclosure layout and estimation of the minimum required enclosure size.

An initial design approach involved cutting an opening in the front of the enclosure in order to place the barcode scanner. However, this approach would have required increasing the overall enclosure size, resulting in a less compact enclosure that would have been bulky even for a prototype smart cart system. Eventually, during the search for a suitable enclosure box, a drawer organizer was identified and selected to be modified to house the barcode scanner. The selected enclosure box was transparent, which enabled precise marking of cut locations and dimensions prior to cutting. This reduced the risk of incorrect cuts and resulted in accurate alignment of the peripheral openings.

Additional openings were created in the enclosure box to support ventilation, which is particularly important for the Raspberry Pi to prevent overheating. These openings also allowed external routing of the barcode scanner cable.

After all modifications were completed, the enclosure was painted black to conceal the internal wiring. Finally, with the exception of the barcode scanner, all hardware components were mounted using screws or hot glue, ensuring stable fixation.



Figure 5. *Physical enclosure design process and final result.*

6 Software Development Process

6.1 Development Methodology

The software development process in Phase B followed an iterative and incremental approach. At each iteration, the current state of the software was tested early rather than deferred to the end of development, enabling timely identification of issues, informed design refinements, and the incremental addition of new functional capabilities. This approach reduced integration risks.

6.1.1 Task Division

Development responsibilities were divided between the two developers, with each focusing on different parts of the system. One developer focused on smart cart-related components, and the other on non-smart cart components, as shown below in Table 1. This division reduced the likelihood of merge conflicts, and made it easier to reason about design decisions.

Table 1. *Division of Development Responsibilities.*

Developer A	Developer B
Controllers	Database
Smart Cart Automation	Backend
Kiosk UI	List-Creation UI

Although responsibilities were clearly divided, the development process also included regular peer review of each other’s code. These reviews helped identify deviations from the intended plan occurred unintentionally and surface potential improvements, contributing to higher code quality.

6.1.2 Testing

Testing was performed continuously throughout Phase B in accordance with the verification test plan defined in Phase A. Each developer validated the components under their responsibility. To support early validation, mock data and AI-generated lightweight user interfaces, were employed to quickly verify functionality of logic modules without the need to implement the final user interfaces.

6.1.3 Collaboration

Collaboration focused primarily on integration stages between the Kiosk UI and the backend, during which components developed independently were connected and validated together, and issues were resolved as needed.

In addition, crafting the voice assistant on VAPI servers was carried out jointly. This component required close collaboration, as it involved refining the system prompt and defining effective tool descriptions to achieve reliable and accurate behavior. Working together on this part of the system improved the overall quality of the voice assistant behavior, as each developer contributed complementary ideas and perspectives.

6.2 Engineering Adjustments

During Phase B, the system was implemented based on the conceptual design and architectural plan defined in Phase A. However, as development progressed, several adjustments and refinements were introduced relative to the original plan. These changes emerged from practical implementation constraints and insights gained.

6.2.1 Architectural Adjustments

- **Separation of the List-Creation UI and the Kiosk UI**

In the original plan, the requirement of centrally maintaining the entire system led to the decision to merge the List-Creation UI and the Kiosk UI into a single project, which would be deployed once. During development, this approach was reconsidered, due to these factors:

1. Although the user interfaces could be centrally deployed, this alone does not enable centralized system maintenance, as the local controllers running on the Raspberry Pi may require separate updates. Therefore, the requirement of maintaining the entire system centrally is not fully satisfied with this approach, and an alternative approach is presented in Section 6.3.3.
2. Deploying the Kiosk UI introduced additional complexity in the interaction between the browser-based interface and the local controllers, due to browser security constraints.
3. Deploying the Kiosk UI on a server would introduce unnecessary dependency on its availability, creating an additional point of failure in the system.
4. Separation better aligned with separation-of-concerns principles. A list creation component should more naturally integrated into the supermarket's existing website rather than being tightly coupled to the smart cart system itself.

- **Integration of an Additional External LLM Service**

During the planning stage, the LLM integrated within VAPI was expected to select alternative products and explain them to the user. During implementation, it became evident that VAPI does not support producing structured outputs during a conversation, which are required for presenting alternatives in the UI. Consequently, an external LLM service is integrated via API in the backend to generate structured alternative selections, while VAPI is responsible only for providing the spoken explanation.

6.2.2 Usability Adjustments

- **Reposition of Main Navigation Bar**

During the planning stage, the envision was that the navigation bar will be placed to the top of the screen. During implementation, this layout was found unsuitable for the size of the display and its 16:9 aspect ratio, as it reduced the available vertical space for the dashboard. Consequently, the navigation bar is positioned to the left side of the interface.

- **Separation of Shopping List from Cart Contents**

In the conceptual design, products from the shopping list were intended to appear within the cart view as semi-transparent entries, indicating products that were planned to be purchased but not yet added to the cart. During implementation, this approach was considered potentially confusing for users, and therefore the shopping list is eventually separated into a dedicated panel, positioned alongside the cart panel.

6.2.3 Behavior Adjustments

- **Modification of Potentially Forgotten Products Suggestions**

It was envisioned that users would first mark suggested forgotten products, which would then appear in the cart as semi-transparent entries awaiting. For the same reason we separated the shopping list from cart contents, the implemented behavior allows users to directly specify the desired quantity for each suggested product within the suggestion modal.

- **Relaxation of Checkout Restrictions**

During planning, checkout was intended to be disabled until all products in the shopping list were either added to the cart or removed. This behavior was not implemented, as it could lead users to interpret the restriction as a system malfunction. Instead, the system allows checkout at any time while presenting a warning indicating that not all products from the shopping list have been added to the cart.

- **Modification of Personalization Control**

The initial plan was that the voice assistant would ask follow-up questions when inferring potential allergies or dietary needs from the user inquiries. This approach was reassessed due to both increased development complexity and the risk of leading to higher operational costs since in that scenario interactions are longer. Instead, personalization is designed as a user-controlled mechanism, in which users explicitly specify the allergies or dietary needs they wish to be alerted about, and can also specify which alerts they wish to stop receiving.

6.3 Implementation Overview

6.3.1 Database Design and Indexing Strategy

The database includes persistent data and temporary, short-lived data generated during system operation. Persistent collections store core entities such as users, products, product categories and their locations, users shopping lists, and purchase-tracking information, while temporary collections are used for operational purposes, including cart session backups and one-time password (OTP) verification during authentication. Cart sessions are periodically synchronized to the database to enable recovery in case of failure, and OTP records are removed once they expire or are successfully used.

Although an in-memory data storage such as Redis would be well suited for managing temporary data and caching frequently accessed items, it was intentionally not introduced in order to avoid additional architectural complexity and deployment overhead. Instead, the system relies on database-level indexing to ensure efficient

access patterns. Indices were created on frequently queried fields, such as product barcodes, enabling fast retrieval without the need for a dedicated caching layer at the current scale.

Product data was populated using information collected from Israeli supermarket websites, resulting in a dataset of over 100 products with detailed attributes. This volume was sufficient to validate that shopping list creation remain responsive.

6.3.2 Touch-Optimized Kiosk User Interface

Since the system does not rely on an operating system natively designed for touchscreen interaction, the kiosk user interface was explicitly designed to be touch-optimized at the application level. To deliver a user experience consistent with modern touch-based applications, the user interface logic was designed to reinterpret mouse drag events as touch-like gestures. Specifically, dragging a scrollable component upward with the mouse results in downward scrolling, and vice versa, effectively emulating natural touch interaction. The mouse cursor and default scroll bars were hidden to prevent distraction.

6.3.3 Automated Operation and Maintenance

All core services are configured to start automatically on boot and to recover from failures without user intervention, ensuring continuous operation after crashes or power interruptions.

For maintainability and debugging purposes, runtime logs are persistently stored. However, due to the constrained storage capacity of the cart hardware, an automated cleanup mechanism was introduced to periodically remove logs older than three days. This prevents uncontrolled storage growth while preserving recent diagnostic information. In addition, software updates are managed through a centralized mechanism that pulls changes from a private source code repository, ensuring consistent versions across deployed carts. Update operations and maintenance-related tasks are intentionally scheduled to occur during nighttime hours, when the system is not in active use, in order to avoid disrupting the shopping experience.

6.3.4 List-Based Shopping Guidance

To support list-based shopping guidance, a simplified model of a real supermarket was documented and represented as a grid-based layout stored in JSON format, shared by both the frontend and backend. A key design consideration was flexibility: since product placement may change over time, locations were not hardcoded but instead stored in the database per product category. Based on this layout, a breadth-first search (BFS) algorithm was used to precompute distances between all walkable grid coordinates, producing a complete distance matrix that is generated once for each new supermarket layout prior to deployment. Route optimization is performed using Google OR-Tools, which applies Traveling Salesman Problem (TSP) algorithms to compute a near-optimal pickup order based on the distances between product locations derived from their associated categories. Given the large size of the full distance matrix, NumPy is used to efficiently filter only the relevant distances corresponding to products in the user's shopping list. The resulting optimized pickup order is stored with the shopping list in the database, and the supermarket grid and route are visually rendered in the kiosk interface using CSS.

6.3.5 Potentially Forgotten Products Suggestions

While recommender systems commonly rely on machine learning models, this capability was implemented using a rule-based approach.

While machine learning models are commonly used in recommender systems, here we established a rule-based but smart approach. The core idea is to compute the average number of days between purchases for each product, estimate the next expected purchase date by adding this average interval to the product's last purchase date, and suggest products within a small-time window around this estimated date, accounting for the fact that users may not shop on the exact estimated day.

To make this approach effective in real-world scenarios, the following considerations were addressed:

- **Cold-start** - Product suggestions are generated only after a product has been purchased at least three times, ensuring sufficient historical data to compute a reliable average interval.
- **Infrequently Purchased Products** - Products with long purchase intervals are difficult to predict accurately. Therefore, only frequently-bought products, defined as products with an average purchase interval of less than 30 days are considered.
- **User Absence and Product Abandonment** - If a user has not visited the supermarket for an extended period, the system assumes temporary absence rather than product abandonment. In such cases, all frequently purchased products are suggested when the user returns.

6.3.6 Voice Assistant

The development process of the voice assistant involved iterative refinement of the system prompt and tool descriptions. Initial attempts focused on prompt precision and example-based guidance. However, these alone did not consistently produce reliable behavior. As a result, different language models were evaluated, starting from the lowest-cost and fastest option, GPT-4.1 Nano, and progressing to GPT-4.1 Mini, which provided improved reasoning and response quality at the expense of slightly reduced response speed, while still maintaining an acceptable cost level.

When the voice assistant was built, the following considerations were addressed:

- **Irrelevant Input** – The assistant was designed to terminate the interaction when he is requested to answer questions that fall outside his scope. In such cases, the user is informed that he cannot violate this policy.
- **User Silence** – To prevent voice sessions from remaining open indefinitely without user interaction, which may incur unnecessary operational costs, a timeout mechanism was introduced. If no speech activity is detected for a predefined period, the interaction is automatically terminated.
- **Context Management** – The VAPI voice assistant maintains conversational context only within a single interaction. Consequently, context is lost between interactions. To mitigate this limitation, the most recent five chat messages are sent to the voice assistant with each new interaction, allowing to preserve short-term conversational continuity.

Structured data is sent to the voice assistant at the beginning of each interaction, including the user's phone number, recorded allergies and dietary needs, and the barcode of the most recently added product. This enables some of the supported voice assistant capabilities:

- **Answering Questions** - Using the provided barcode, a configured tool is invoked to retrieve product data, including ingredients and nutritional values, enabling the voice assistant to answer the questions.
- **Suggesting Alternatives** - Using the provided barcode together with the user's recorded allergies and dietary needs, a configured tool is invoked to retrieve products from the same category (e.g., the mayonnaise category) while filtering out products that conflict with the user's allergies or dietary requirements. The remaining candidates are then provided to Gemini (as described in Section 6.2.1), which selects suitable alternatives based on the user's stated requirement, and provides an explanation.
- **Personalization** - Using the user's phone number together with stated allergies or dietary needs that, a configured tool is invoked to add or remove these preferences from the user profile. Based on these updates, automatic alerts related to allergy or dietary conflicts are enabled or disabled accordingly.

Automatic alerts cannot be triggered directly from within the VAPI voice assistant, as generating a spoken response requires establishing an explicit interaction session. Consequently, these alerts are implemented using only a rule-based mechanism. However, the alert audio is recorded using the same voice as the assistant to maintain a consistent user experience and reinforce the perception of a unified intelligent system.

When location guidance is requested, the voice assistant invokes another tool that resolves whether the user's query refers to a product category or a specific product. Category queries are matched directly and return the corresponding category location. Otherwise, product names are searched using flexible text matching, and the best-matching product is selected. If the identified product is unavailable due to supply constraints or does not exist, this availability information is returned to the assistant, ensuring availability awareness and preventing location guidance for unavailable products.

6.4 Development Challenges

6.4.1 Technical Challenges

Most of the technical challenges encountered during development were related to the integration of VAPI:

- **VAPI Control Message Reliability and Latency Issues** - VAPI exposes a stream of control messages that provide structured information about events during a conversation. However, since VAPI is still an evolving platform, some expected messages were unreliable or missing in practice. In particular, the documented mechanism for receiving tool execution results did not function as expected, requiring direct communication with VAPI support, which suggested integration of an alternative, undocumented control message that exposed the required tool results. Additionally, transcription events often exhibited high latency, which was mitigated by using the model's generated output, typically closely aligned with the spoken transcription.

- **Echo Issues** - During early development, before we received the microphone array with the echo-cancelling algorithm, the voice assistant's audio output was captured by the microphone, causing the assistant to receive its own speech and enter a response loop. To mitigate this behavior, the microphone is automatically muted while the assistant is speaking. This solution prevents feedback loops but disables the user's ability to interrupt the assistant mid-response.

6.4.2 Time Management Challenges

While the project was expected to require a substantial time investment, the cumulative impact of unforeseen technical and algorithmic design challenges led to a longer development process than anticipated. As a result, the project reached completion very close to the presentation time, requiring some evaluation activities to be postponed and conducted later, close to the final submission deadline.

7 Results

The system was successfully implemented to meet all core requirements defined in Phase A, including those that were adjusted during Phase B. All system components were fully integrated and deployed, resulting in a stable, end-to-end smart cart solution that received positive feedback during the presentation.

8 Evaluation and Conclusions

8.1 Evaluation

Table 2. *System Evaluation Summary.*

Aspect	Evaluation Method	Requirement	Result
Voice Assistant Response Latency	Measurement via VAPI analytics and observation during testing	$\leq 3 \text{ Seconds}$	✓
Concurrent Voice Assistant Sessions	Parallel execution on three separate client machines	$\geq 3 \text{ Carts}$	✓
Voice Assistant Intent-Recognition Accuracy	Analysis of interaction logs collected during internal testing and user trials, measuring whether the assistant correctly identified and executed the intended user request.	$\geq 90\% \text{ of Inquiries}$	94%
SUS Score	SUS questionnaire completed by volunteers	$\geq \text{Average of } 68$	89.5

Moreover, additional feedback collected alongside the SUS evaluation indicated that most participants found the pickup route guidance and voice-based product location requests to be the most useful features.

In addition, a representative cost breakdown of a single voice assistant interaction is presented in Figure 6, with similar cost distributions observed across multiple sessions.

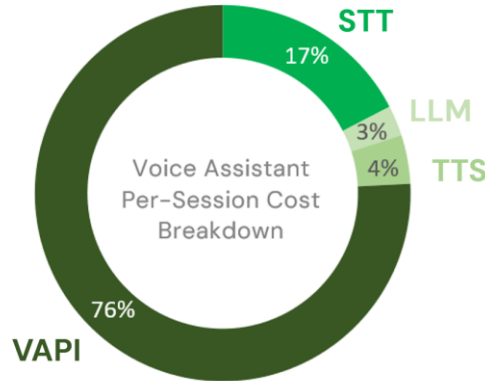


Figure 6. *Representative cost breakdown of a voice assistant interaction session.*

8.2 Conclusions

1. The voice assistant exhibited low response latency due to the selection of a lightweight language model. Despite the VAPI servers being located in the United States, geographically distant from the test country, interaction remain responsive, ensuring that the assistant adds a practical value to the system.
2. The project demonstrated that careful prompt engineering enables the use of lightweight language models, achieving low interaction costs without compromising response quality or overall intent-recognition accuracy. The remaining reduction in intent-recognition accuracy is primarily attributed to background noise affecting STT transcription. In such cases, the model typically requests clarification rather than executing an incorrect action.
3. Concurrent voice assistant sessions, a realistic usage scenario, are supported through the choice of VAPI, which allows up to ten parallel conversations for non-enterprise users. However, the cost analysis indicates that VAPI constitutes the dominant portion of per-session costs, while STT, LLM, and TTS components contribute relatively marginally. This suggests that, for large-scale deployment, a custom voice processing pipeline could significantly reduce operational costs and improve cost feasibility.
4. The SUS score of 89.5 indicates a high level of perceived usability. Based on specific questionnaire responses and the additional feedback, the score did not reach the maximum value due to the richness of system features, which requires a learning period, as well as minor frictions, such as limited quantity input options that rely on repeated scanning or incremental controls instead of direct numeric input via a keypad.

9 Lessons Learned

1. This project highlighted the importance of the engineering process during Phase A, which significantly simplified development and reduced overall implementation time. Having a clear architectural and functional plan from the outset enabled efficient progress. However, some algorithmic components, such as the logic for the potentially forgotten product suggestions, were postponed to the implementation phase despite being conceptually suitable for earlier design, and required an amount of time to reason about. This experience emphasized that even components perceived as minor should be addressed during the engineering phase.
2. The project demonstrated the significant role of AI-assisted development in modern software engineering and learning processes. As a team consisting of two students, assistance from AI tools enabled the implementation of a system with complexity comparable to industry-level projects. Without such assistance, it is unlikely that the project could have been completed within the given timeframe, and the number of system capabilities would likely have been reduced. This experience highlights AI as a practical enabler for rapid development and accelerated learning.

10 References

- [1] Wang, S., Ye, Y., Ning, B., Cheah, J. H., & Lim, X. J. (2022). Why do some consumers still prefer in-store shopping? An exploration of online shopping cart abandonment behavior. *Frontiers in Psychology*, 12, 829696. <https://doi.org/10.3389/fpsyg.2021.829696>
- [2] Van Riel, A. C., Semeijn, J., Ribbink, D., & Bomert-Peters, Y. (2012). Waiting for service at the checkout: Negative emotional responses, store image and overall satisfaction. *Journal of service management*, 23(2), 144-169. <https://doi.org/10.1108/09564231211226097>
- [3] Bloemer, J., & De Ruyter, K. (1998). On the relationship between store image, store satisfaction and store loyalty. *European Journal of marketing*, 32(5/6), 499-513. <https://doi.org/10.1108/03090569810216118>
- [4] Schultz, C. D., & Zacheus, P. (2025). Smart shopping carts in food retailing: Innovative technology and shopping experience in stationary retail. *Journal of Consumer Behaviour*, 24(1), 436-454. <https://doi.org/10.1002/cb.2426>
- [5] Bohns, V. K., & Flynn, F. J. (2010). “Why didn’t you just ask?” Underestimating the discomfort of help-seeking. *Journal of Experimental social psychology*, 46(2), 402-409. <https://doi.org/10.1016/j.jesp.2009.12.015>
- [6] Fan, H., & Poole, M. S. (2006). What is personalization? Perspectives on the design and implementation of personalization in information systems. *Journal of Organizational Computing and Electronic Commerce*, 16(3-4), 179-202. <https://doi.org/10.1080/10919392.2006.9681199>
- [7] Chen, J., Liu, Z., Huang, X., Wu, C., Liu, Q., Jiang, G., ... & Chen, E. (2024). When large language models meet personalization: Perspectives of challenges and opportunities. *World Wide Web*, 27(4), 42. <https://doi.org/10.1007/s11280-024-01276-1>

- [8] Warren, C. M., Jiang, J., & Gupta, R. S. (2020). Epidemiology and burden of food allergy. *Current allergy and asthma reports*, 20(2), 6. <https://doi.org/10.1007/s11882-020-0898-7>
- [9] Food Allergy Research & Education. (n.d.). *Facts and statistics*.
<https://www.foodallergy.org/resources/facts-and-statistics>
- [10] Neuhouser, M. L. (2019). The importance of healthy dietary patterns in chronic disease prevention. *Nutrition research*, 70, 3-6. <https://doi.org/10.1016/j.nutres.2018.06.002>
- [11] Leidy, H. J., Clifton, P. M., Astrup, A., Wycherley, T. P., Westerterp-Plantenga, M. S., Luscombe-Marsh, N. D., ... & Mattes, R. D. (2015). The role of protein in weight loss and maintenance. *The American journal of clinical nutrition*, 101(6), 1320S-1329S.. <https://doi.org/10.3945/ajcn.114.084038>
- [12] World Health Organization Regional Office for Europe. (2021, November 4). *Plant-based diets and their impact on health, sustainability and the environment: A review of the evidence* (WHO Reference No. WHO/EURO:2021-4007-43766-61591). Copenhagen: WHO Regional Office for Europe.
<https://www.who.int/europe/publications/i/item/WHO-EURO-2021-4007-43766-61591>
- [13] Deakin, T. A. (2011). Consumers find food labels confusing and too small to read. *Practical Diabetes International*, 28(6), 261-264c. <https://doi.org/10.1002/pdi.1611>
- [14] Applegate, D. L., Bixby, R. E., Chvátal, V., & Cook, W. J. (2011). The traveling salesman problem: a computational study. In *The Traveling Salesman Problem*. Princeton university press.
<https://doi.org/10.1515/9781400841103>
- [15] Davendra, D. (Ed.). (2010). *Traveling salesman problem: Theory and applications*. BoD–Books on Demand. <https://doi.org/10.5772/547>
- [16] Wu, J., Gan, W., Chen, Z., Wan, S., & Yu, P. S. (2023, December). Multimodal large language models: A survey. In *2023 IEEE International Conference on Big Data (BigData)* (pp. 2247-2256). IEEE.
<https://doi.org/10.48550/arXiv.2311.13165>
- [17] Li, J., Xu, J., Huang, S., Chen, Y., Li, W., Liu, J., ... & Dai, G. (2024). Large language model inference acceleration: A comprehensive hardware perspective. *arXiv preprint arXiv:2410.04466*.
<https://doi.org/10.48550/arXiv.2410.04466>
- [18] Samsi, S., Zhao, D., McDonald, J., Li, B., Michaleas, A., Jones, M., ... & Gadepally, V. (2023, September). From words to watts: Benchmarking the energy costs of large language model inference. In *2023 IEEE High Performance Extreme Computing Conference (HPEC)* (pp. 1-9). IEEE.
<https://doi.org/10.48550/arXiv.2310.03003>
- [19] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... & Polosukhin, I. (2017). Attention is all you need. *Advances in neural information processing systems*, 30.
<https://doi.org/10.48550/arXiv.1706.03762>

- [20] Marvin, G., Hellen, N., Jjing, D., & Nakatumba-Nabende, J. (2023, June). Prompt engineering in large language models. In *International conference on data intelligence and cognitive informatics* (pp. 387-402). Singapore: Springer Nature Singapore. https://doi.org/10.1007/978-981-99-7962-2_30
- [21] Rubei, R., Moussaid, A., Di Sipio, C., & Di Ruscio, D. (2025). Prompt engineering and its implications on the energy consumption of Large Language Models. *arXiv preprint arXiv:2501.05899*.
<https://doi.org/10.48550/arXiv.2501.05899>
- [22] Hoy, M. B. (2018). Alexa, Siri, Cortana, and more: an introduction to voice assistants. *Medical reference services quarterly*, 37(1), 81-88. <https://doi.org/10.1080/02763869.2018.1404391>
- [23] Yu, D., & Deng, L. (2016). *Automatic speech recognition* (Vol. 1). Berlin: Springer.
<https://doi.org/10.1007/978-1-4471-5779-3>
- [24] Tan, X., Qin, T., Soong, F., & Liu, T. Y. (2021). A survey on neural speech synthesis. *arXiv preprint arXiv:2106.15561*. <https://doi.org/10.48550/arXiv.2106.15561>
- [25] Chan, S., Fu, S., Li, J., Yao, B., Desai, S., Prpa, M., & Wang, D. (2024). Human and llm-based voice assistant interaction: An analytical framework for user verbal and nonverbal behaviors. *arXiv preprint arXiv:2408.16465*. <https://doi.org/10.48550/arXiv.2408.16465>
- [26] Lu, J., Wu, D., Mao, M., Wang, W., & Zhang, G. (2015). Recommender system application developments: a survey. *Decision support systems*, 74, 12-32. <https://doi.org/10.1016/j.dss.2015.03.008>
- [27] Lü, L., Medo, M., Yeung, C. H., Zhang, Y. C., Zhang, Z. K., & Zhou, T. (2012). Recommender systems. *Physics reports*, 519(1), 1-49. <https://doi.org/10.1016/j.physrep.2012.02.006>

Appendices

User Manual

1. Introduction

This user manual provides instructions and guidance for operating the smart cart system. A smart cart system maintains a real-time list of products added to the cart and continuously displays the accumulated cost throughout the shopping session, enabling users to pay directly at the cart and avoid standing in line.

In addition to its core functionality, this specific smart cart system provides advanced assistance features designed to reduce further the shopping time, including:

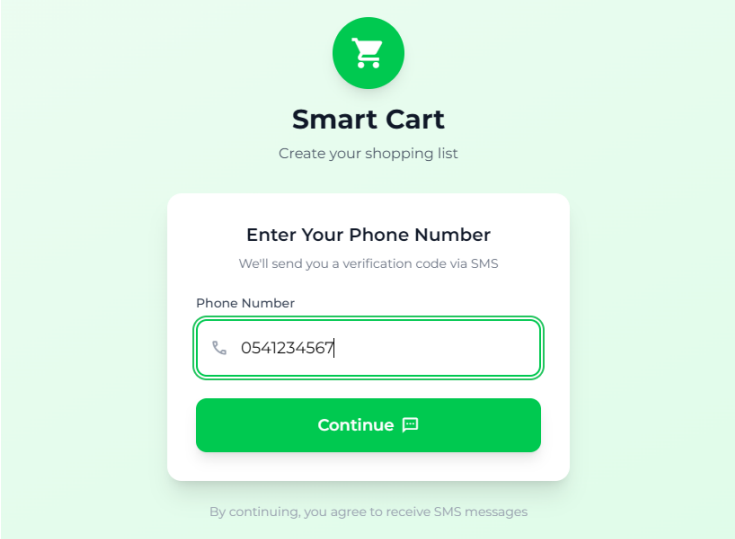
- **AI-Based Voice Assistant**
Enables natural voice interaction for checking ingredients and nutritional values, detecting dietary conflicts, suggesting suitable alternatives, and locating products within the store.
- **List-Based Shopping Guidance**
Supports loading a shopping list and guides the user through an efficient in-store route while ensuring all listed products are collected.
- **Potentially Forgotten Product Suggestions**
Provides personalized recommendations at checkout based on historical purchasing behavior, reducing the likelihood of forgetting to purchase products.

2. System Usage

2.1. Shopping List Creation and Editing

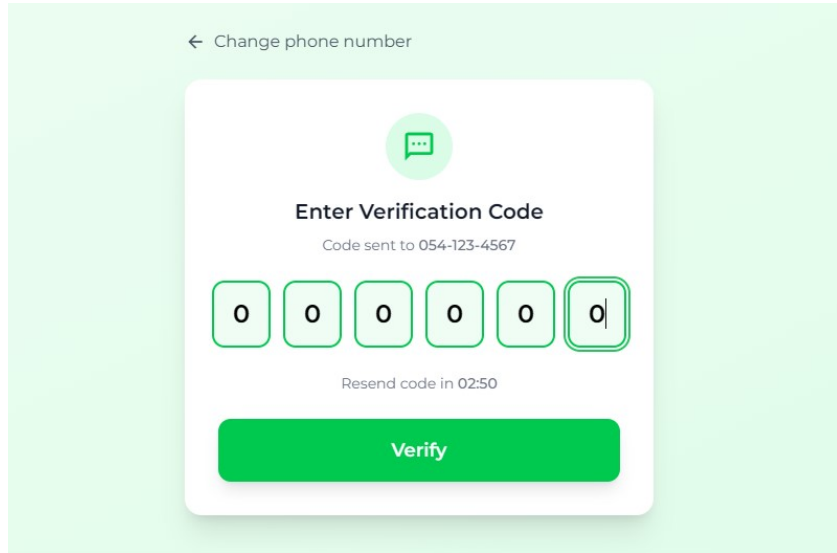
Creating a shopping list in advance is recommended but not required. To use the smart cart without creating a shopping list in advance, or to edit the shopping list before the shopping session, proceed to Section 2.2.

- 2.1.1. To create a shopping list, the user should access the web application [here](#) using a personal device (computer, tablet, or smartphone). Enter a phone number in the designated field, then press “Continue”.

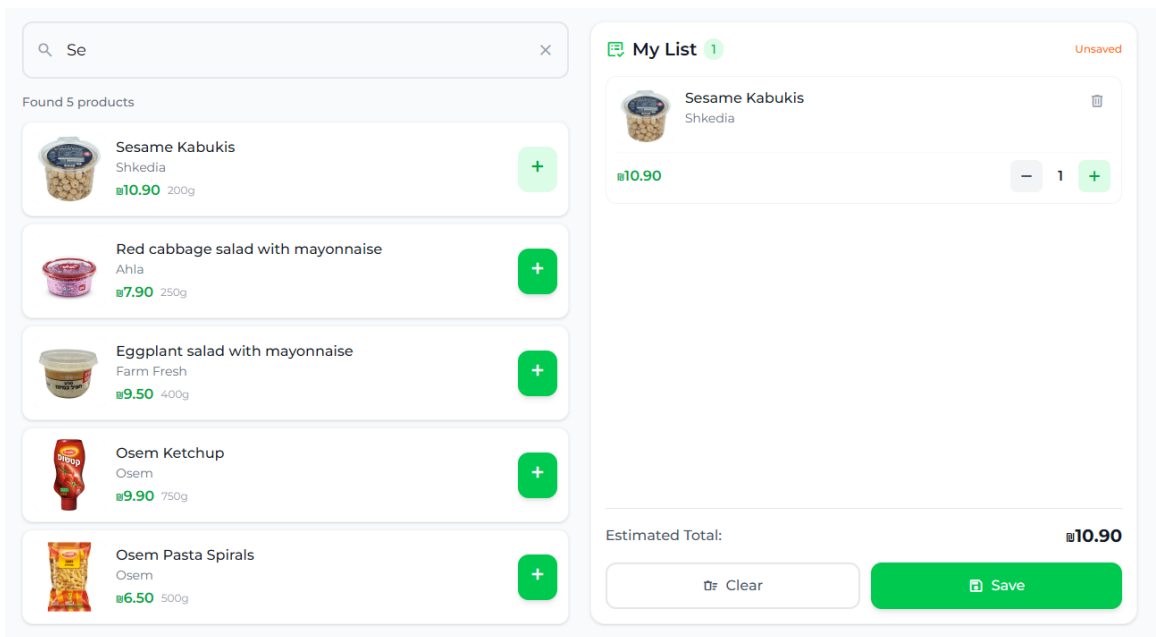


The image shows a mobile application interface for a 'Smart Cart'. At the top, there is a green circular icon with a white shopping cart symbol. Below it, the text 'Smart Cart' is displayed in bold, followed by 'Create your shopping list' in a smaller font. The main content area is a white card with a green border. Inside the card, the heading 'Enter Your Phone Number' is followed by the subtext 'We'll send you a verification code via SMS'. Below this, there is a label 'Phone Number' and a text input field containing the number '0541234567'. A green button with the text 'Continue' and a small speech bubble icon is positioned below the input field. At the bottom of the card, a small disclaimer reads 'By continuing, you agree to receive SMS messages'.

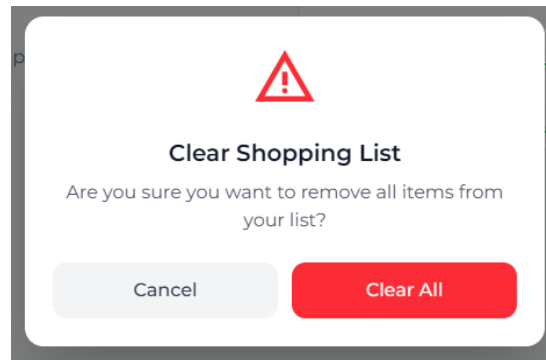
- 2.1.2. Enter the one-time password (OTP). **Due to free-tier limitations, the OTP is currently fixed as 000000 for all users, and no SMS message is sent.** The interface will indicate the phone number to which the code is supposedly sent. If an incorrect phone number was entered, select **“Change phone number”** and re-enter the phone number. Otherwise, select **“Verify”** to continue. The verification must be completed within **3 minutes**. If the code is not received within this time period, select **“Resend Code”** to request a new code.



- 2.1.3. After successful authentication, the interface is displayed. If an existing shopping list was created previously and requires editing or deletion, proceed to Section 2.1.7. This interface allows searching for products using the search bar and adding them to the shopping list. Products added to the list appear in the **“My List”** panel, where the current contents of the shopping list, and total price, are shown.



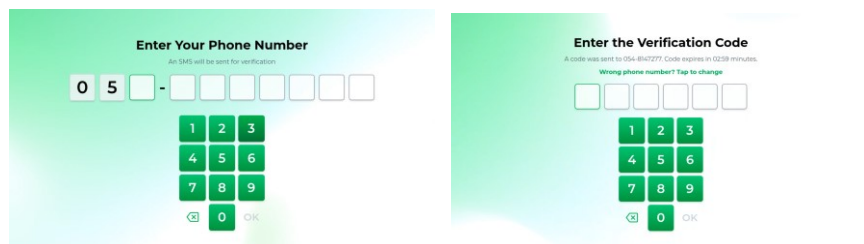
- 2.1.4. To increase or decrease the quantity of a product, press the ‘+’ or ‘-’ buttons on the product card, respectively. To remove a product completely, press the **trash** icon.
- 2.1.5. Once all desired products have been added with their quantity, press the **“Save”** to save the shopping list.
- 2.1.6. To log out, press the **“Logout”** button located in the upper-right corner.
- 2.1.7. To delete an existing shopping list, press **“Clear”** located at the bottom-left of the **“My List”** panel. A confirmation prompt is displayed. Select **“Clear All”** to complete the deletion.



- 2.1.8. If the shopping list is not cleared and modifications are required, product quantities can be adjusted as described in previous sections, and additional products can be added. After completing the changes, press **“Save”**.

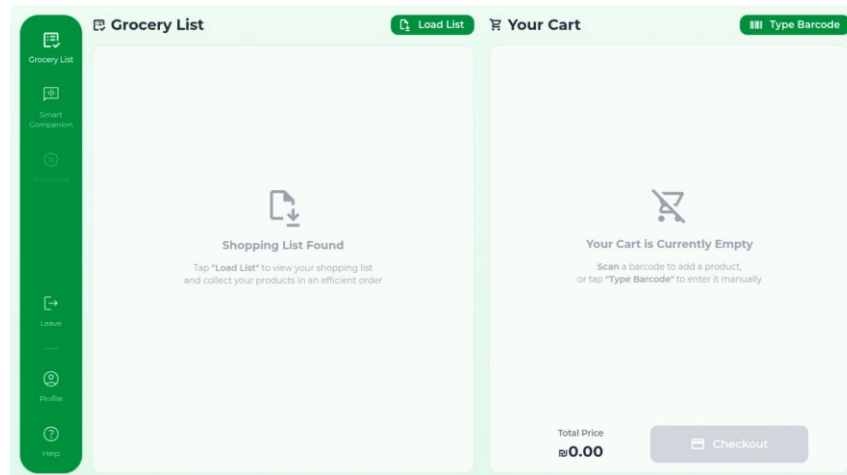
2.2. Smart Cart Authentication

Authentication on the smart cart is performed using the same phone number and one-time password (OTP) mechanism described in Section 2.1. The process is identical and therefore not repeated here.



2.3. Smart Cart Navigation

Once authentication is completed, the main dashboard is displayed. The dashboard is divided into two primary panels. The cart content panel, located on the right side, remains static throughout the shopping session and continuously displays the current cart contents and total price. The left panel is dynamic and changes according to the selected functionality. Specifically, selecting “**Grocery List**” or “**Smart Companion**” updates the content of this panel accordingly.



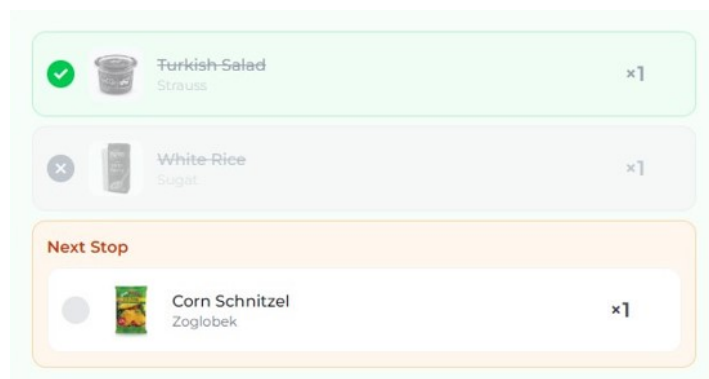
2.4. Shopping List Tracking and Navigation

If a shopping list was created in advance, the “**Load List**” button is enabled in the grocery list panel. If no shopping list was created, proceed to Section 2.5.

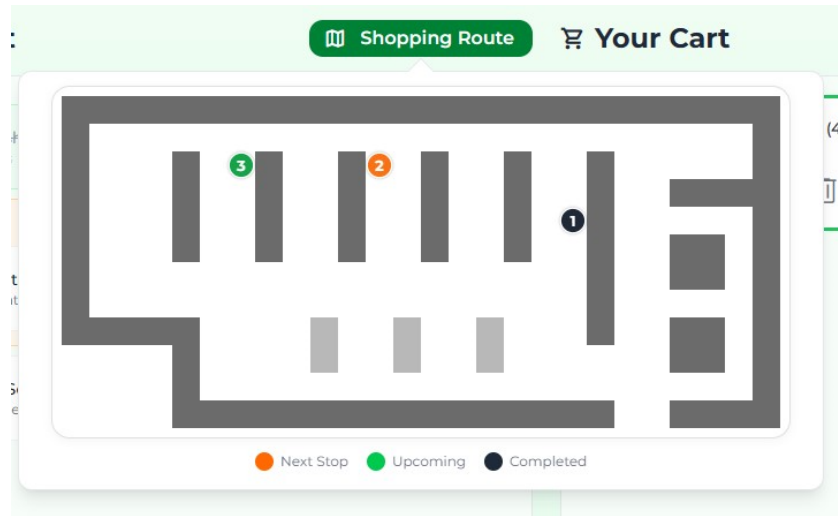
2.4.1. Select “**Load List**” to load the shopping list.

2.4.2. After loading the list, the button changes to “**Shopping Route**”, and the full shopping list is displayed in the panel.

2.4.3. Products that have already been collected are marked in green with a checkmark icon. The next products to be collected along the shopping route are highlighted in orange. Products can also be removed from the shopping list by tapping their corresponding status icon.

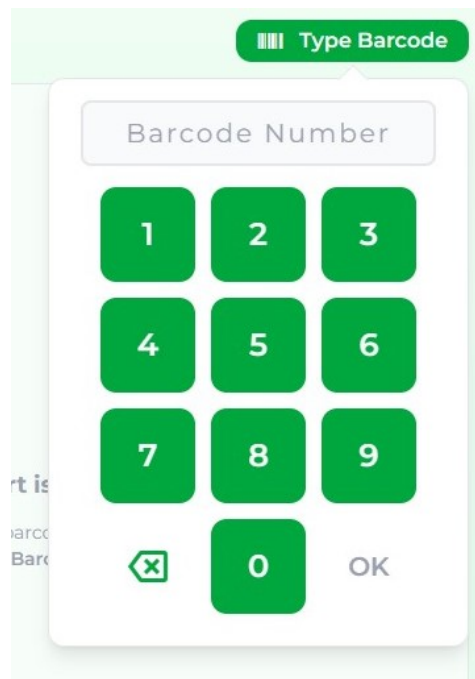


2.4.4. To display the shopping route, select the “**Shopping Route**” button. The same color scheme is applied, indicating collected products in **green**, the next products to be collected in **orange**, and completed or deleted products in **black**.



2.5. Adding Products to the Cart and Adjusting Quantity

2.5.1. Products can be added to the cart either by selecting “**Type Barcode**” for manual entry or by using the barcode scanner located on the right side of the enclosure. To scan a product, remove the scanner from its holder, aim it at the product barcode, and press the scanner button.



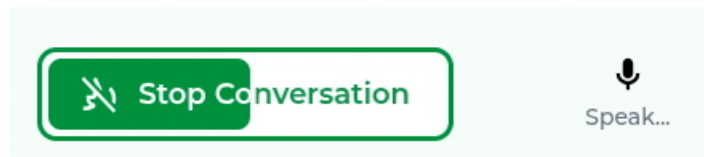
2.5.2. After a product is added to the cart, it is highlighted with a **green outline**, indicating that the product is available for inquiries and alternative suggestions via the Smart Companion.



2.5.3. To adjust the quantity of a product, use the ‘**up**’ and ‘**down**’ buttons displayed on the product card. To delete the product completely, use the ‘**trash**’ icon. Changes are applied immediately and reflected in the cart and total price.

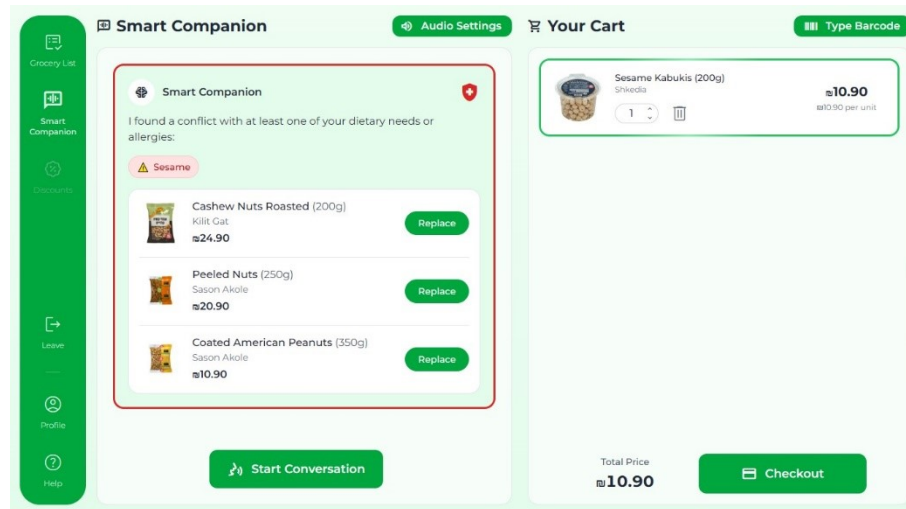
2.6. Using the Smart Companion

2.6.1. To use the **Smart Cart Companion**, navigate to the side panel and press the “**Start Conversation**” button. The system will automatically connect you to the assistant and prompt you when to speak. A progress bar will appear on the button, indicating the listening duration. If the bar reaches its end without detecting speech, the conversation will automatically terminate.



2.6.2. Ask one of these:

- To receive automatic alerts and personalized alternative product suggestions based on dietary requirements. The currently supported **allergies** are **sesame, peanuts, tree nuts, dairy, eggs, and soy**, and the supported **dietary preferences** are **vegan, vegetarian, gluten-free, lactose-free, and sugar-free**. Multiple allergies may be specified together, and multiple dietary preferences may also be specified together. However, allergies and dietary preferences must be provided **separately** and not combined in a single request.



- Nutritional information and/or ingredient details for product marked with a **green outline**.
- Alternative product suggestions based on specified requirements.
- Displaying product locations. The smart companion accounts for stock availability.

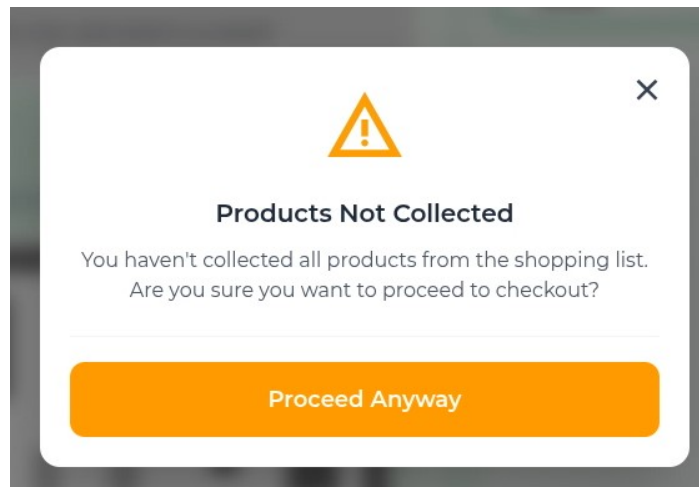
2.6.3. In case irrelevant question are asked, the smart companion will terminate the call.

2.6.4. To terminate the call, select “Stop Conversation”.

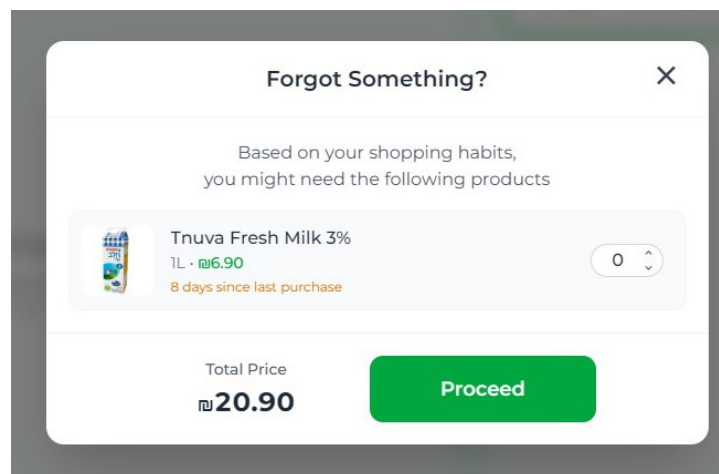
2.7. Checkout

2.7.1. When finished adding products, press the “**Checkout**” button.

2.7.2. If not all products on the shopping list have been added to the cart, a warning is displayed. Select “**Proceed**” to continue.



2.7.3. The system displays suggestions for products that may have been forgotten, based on previous purchasing behavior. Suggested products can be added to the cart if desired.



Maintenance Guide

1. Introduction

This document serves as a comprehensive Maintenance Guide for the Personalized Smart Cart with AI Assistance project. It is intended for system administrators, developers, and maintenance personnel responsible for the deployment, operation, and ongoing upkeep of the system. The guide details the operational environment, installation procedures for custom components, and strategies for future updates and maintenance.

1. System Overview

The Personalized Smart Cart system has several key components that work together:

- **Backend API (FastAPI + MongoDB):** Central service handling data, business logic, and integrations with external services
- **Voice Assistant Controller (Python):** Manages voice interactions via VAPI, runs on Raspberry Pi
- **Barcode Scanner Controller (Python):** Interfaces with barcode scanner hardware, runs on Raspberry Pi
- **Kiosk Frontend (React):** Main user interface on the Smart Cart device
- **List-Creation Frontend (React):** Web-based application for remote shopping list management

All components communicate via standard HTTP/WebSocket protocols.

2. System Requirements

2.1 Hardware Requirements

- **Raspberry Pi** (4 or newer is recommended)
- **Peripherals:** Compatible barcode scanner, microphone, speakers and monitor

2.2 Software Requirements:

- **Operating System:**
 - Production:** Linux-based embedded system.
 - Development:** Windows or Linux (Windows supported for local testing only).
- **Python:** 3.10+
- **Node.js:** versions 18.x or 20.x and above is recommended
- **MongoDB:** cloud service is recommended
- **Git**
- **Pip**

3. Backend Setup & Maintenance:

The Backend API is a FastAPI application that serves as the central service for the entire system

3.1 Installation & Running:

Step 1: Prerequisites

Before installing the backend, for full usability ensure you have:

- Google Gemini API key (for AI alternatives features)
- Twilio API key's (ACCOUNT_SID, AUTH_TOKEN, PHONE_NUMBER) for sms service
- Vapi dashboard configured and ready

Step 2: clone and Install Dependencies

Clone the repository

```
git clone https://github.com/tomerot/personalized-smart-cart-with-ai-assistance
```

Cd into the backend (cd:/Implementation/Backend)

Install all dependencies: `pip install -r requirements.txt`

Step 3: Configure Environment Variables

Create a .env file in Implementation/Backend/ The backend needs the following environment variables configured

```
MONGODB_URI=mongodb://user:password@host:port/database
GEMINI_API_KEY = " Gemini API key"
TWILIO_ACCOUNT_SID = "Twilio SID"
TWILIO_AUTH_TOKEN = "Twilio Token"
TWILIO_PHONE_NUMBER= "Twilio Phone Number"
```

For production, set these as environment variables in your hosting platform (never upload the .env)

Step 4: Start the Backend

If you are working locally then ensure you're on the project backend folder and run:

```
uvicorn main:app --reload
```

The backend should now be accessible at <http://localhost:8000>

3.2 Structure:

The backend is a FastAPI application that serves as the central API for the entire system.

Here's what it contains:

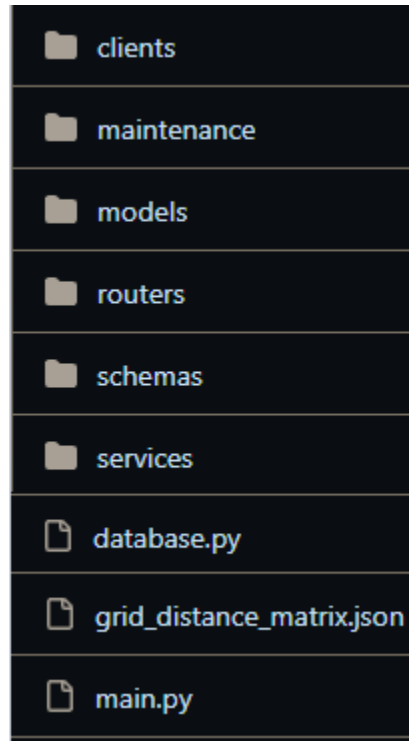


Figure 1: Backend Structure

Clients: Handles communication with external services (Google Gemini for AI, Twilio for SMS)

Models: Defines data structures stored in MongoDB

Routers: Defines API endpoints that frontend applications call

Schemas: Defines Pydantic models for request/response validation

Services: Contains the actual business logic

Database.py: MongoDB connection setup

Main.py: Starts the FastAPI application, sets up database connection, configures CORS

3.3 Maintenance Tasks:

- Periodically update Python libraries to their latest version: from the backend folder run:

```
pip install --upgrade -r requirements.txt
```

IMPORTANT sometimes updating Python libraries can cause errors due to version incompatibility, always make sure those updated are compatible.
- From time to time the supermarket may change its products location, after each time make sure the DB "Categories" Table is updated and run the "calculate_grid_distances.py" in the maintenance folder to update the distance matrix.

4. Frontend Setup & Maintenance:

There are two separate React applications: Kiosk and List-Creation.

4.1 Installation:

Step 1: Clone and Install Dependencies

Clone the repository:

```
git clone https://github.com/tomerot/personalized-smart-cart-with-ai-assistance
```

The installation for both applications is the same: cd into the relevant folder and run:

```
npm install
```

Step 2: Configure Environment Variables

- List-Creation: skip this step
- Kiosk: Create a .env.local file in the Kiosk folder:

```
VITE_API_BASE_URL = http://your-backend-server VITE_WS_VA_URL =  
ws://localhost:8765  
VITE_WS_BS_URL = ws://localhost:8766
```

Step 3: Build

- For development run:

```
npm run dev
```
- For production run:

```
npm run build
```

4.2 Structure:

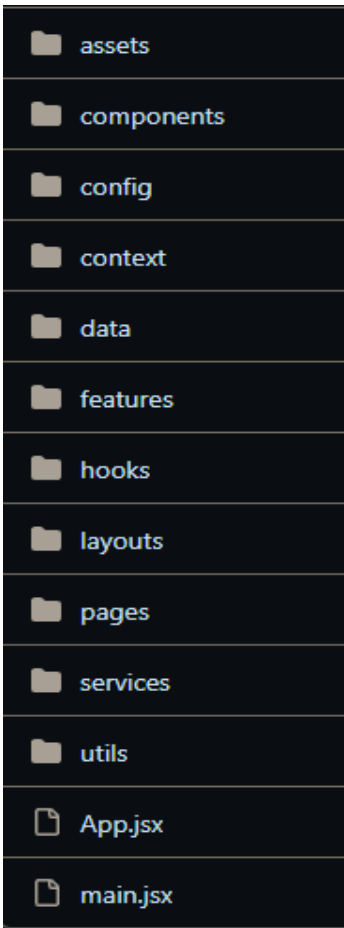


Figure 2: Kiosk Structure

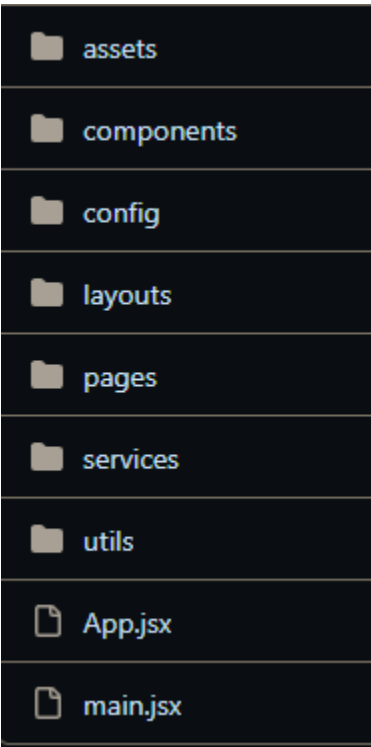


Figure 3: List Creation Structure

- Assets:** Contains static assets for the application.
- Components:** Reusable UI components used throughout the application.
- Config:** Configuration files for the application.
- Context:** React Context providers for global state management.
- Data:** Static data files and data-related utilities.
- Features:** Feature-specific components and hooks
- Hooks:** Custom React hooks that encapsulate reusable logic.
- Layouts:** Layout components that define the overall structure of different pages.

Pages: Top-level components representing distinct application screens.

Services: Modules for interacting with backend APIs and external controllers.

Utils: Utility functions and helper modules.

App.jsx: main application component. Sets up React Router for navigation, provides global context

4.3 Maintenance Tasks:

- Periodically update Dependencies To get latest features and security patches by running: `npm update`

5. Raspberry Pi Deployment:

The Kiosk Frontend (React) is deployed on Raspberry Pi

5.1 Controllers overview

The Raspberry Pi runs two Python controllers that handle hardware and communicate with the Kiosk frontend, both can be found under the `/implementation/controllers` folder

- **Voice Assistant Controller (VA):** Manages voice interactions. Receives audio from Kiosk, sends to VAPI, gets response, plays audio through speakers.
- **Barcode Scanner Controller (BS):** Manages barcode scanner hardware. Reads barcodes from scanner, sends them to Kiosk via WebSocket.

5.2 Controllers Structure

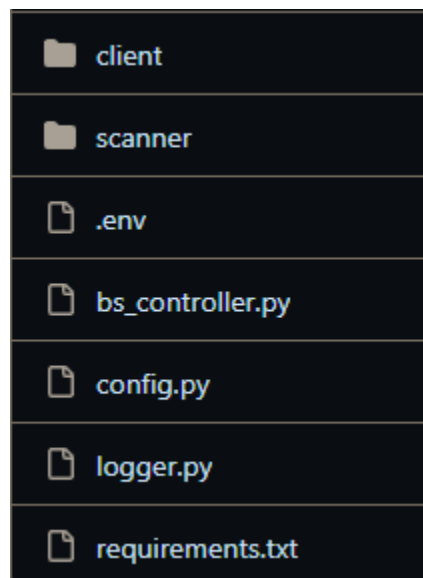


Figure 4: BS Controller Structure

Client: modules responsible for managing client connections via WebSockets.

Scanner: Dedicated modules for interfacing with and managing the barcode scanner hardware

Bs_controller.py: The main entry point for the Barcode Scanner Controller application, orchestrating the scanner input and client communication.

Config.py: Configuration settings for the controller, such as scanner device paths or WebSocket settings.

Logger.py: A utility module for handling logging throughout the controller, ensuring proper error reporting and operational monitoring.

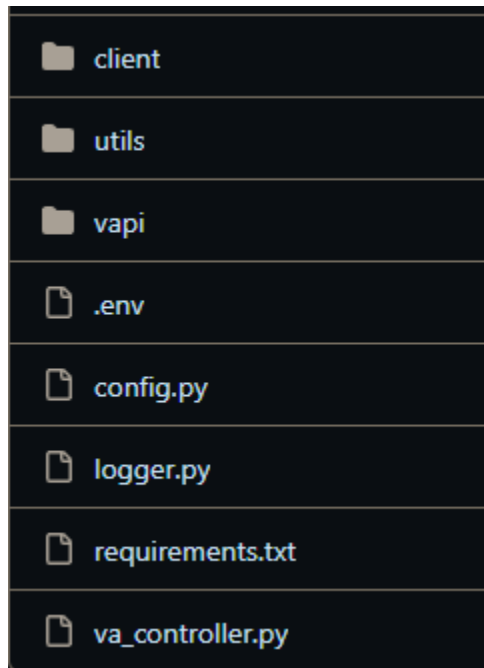


Figure 5: VA Controller Structure

Client: contains module responsible for managing client connections via websocket

Utils: general utility modules used by the controller.

Congif.py: Configuration settings for the controller, including audio device settings, VAPI endpoint URLs, or WebSocket settings.

Logger.py: A utility module for handling logging throughout the controller, ensuring proper error reporting and operational monitoring.

Va_controller.py: The main entry point for the Voice Assistant Controller application, coordinating audio processing, VAPI communication, and client interactions.

5.3 Deployment Directory Structure

The /ImplementationSetup/ folder contains everything needed to deploy on Raspberry Pi:

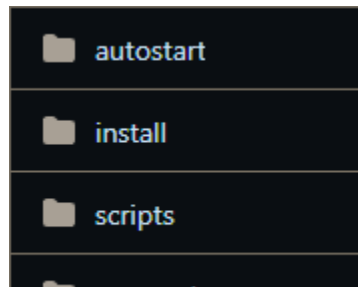


Figure 6: Setup Structure

Autostart: Desktop files for GUI auto-start

Install: Main production installation script

Scripts: Maintenance and updating scripts

Systemd: Service files

5.4 full installation on raspberry Pi

The easiest way to set up the entire system on a fresh Raspberry Pi is to use the provided prod-install.sh script. This script automates all installation steps.

Step 1: Prerequisites

- Fresh Raspberry Pi OS installed
- Internet connection
- Your VAPI API credentials (API key and Assistant ID)

Step 2: Download and run the installation script

- Download the script from the /setup/install/prod-install.sh you can do this by running:
wget
<https://raw.githubusercontent.com/USERNAME/REPOSITORY/main/Implementation/Setup/install/prod-install.sh>
- Make it executable: `chmod +x prod-install.sh`
- And run the script: `./prod-install.sh`

Step 3: Follow the script prompts

The script will:

1. **Generate SSH key** - The script displays your public SSH key. You must add this to GitHub:
 - Go to GitHub → Your repository → Settings → Deploy Keys → Add Deploy Key
 - Paste the key and save it
 - Press Enter in the script to continue
2. **Clone repository** - Downloads your project code
3. **Create Python environments** - Sets up virtual environments for both controllers
4. **Configure credentials** - Asks for:
 - VAPI_API_KEY - Your VAPI API key
 - VAPI_ASSISTANT_ID - Your VAPI Assistant ID
5. **Install systemd services** - Sets up automatic startup of controllers and Kiosk
6. **Setup autostart** - Configures Kiosk to display in fullscreen on boot
7. **Schedule updates** - Creates a daily cron job to check for updates at 3 AM
8. **Reboot** - The script will prompt you to reboot

If everything worked as expected The Smart Cart UI automatically displays in Fullscreen, and all services should run automatically.

5.5 Managing Services

to check the status of all services run:

- `sudo systemctl status kiosk-server.service`
- `sudo systemctl status va-controller.service`
- `sudo systemctl status bs-controller.service`

to start/stop a service use:

- `sudo systemctl stop kiosk-server.service`
- `sudo systemctl start kiosk-server.service`

to Enable/Disable on boot use:

- `sudo systemctl enable kiosk-server.service`
- `sudo systemctl disable kiosk-server.service`

to view service logs use:

- `journalctl -u kiosk-server.service -f`